

26. planner 指令の安全橋渡し

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	7
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 rclpy、rcl、rmw、DDS/RTSPS の層	8
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	8
6.11 rqt_graph、ros2 CLI、rosbag2 をいつ使うか	9
6.12 制御、状態、入力、モデル予測制御 MPC	9
7 関数・クラスを上から順に解説	10
7.1 L17 クラス SpeedCommandBridgeNode	10

7.2	L18 関数 <code>SpeedCommandBridgeNode.__init__</code>	11
7.3	L89 関数 <code>SpeedCommandBridgeNode._on_speed_cmd</code>	14
7.4	L93 関数 <code>SpeedCommandBridgeNode._on_upper_speed_cmd</code>	15
7.5	L96 関数 <code>SpeedCommandBridgeNode._on_drive_mode</code>	16
7.6	L100 関数 <code>SpeedCommandBridgeNode._on_system_state</code>	16
7.7	L104 関数 <code>SpeedCommandBridgeNode._step</code>	17
7.8	L162 関数 <code>main</code>	19
7.9	設定値と入力パラメータ	20
7.10	ROS topic I/O	21
8	処理の流れ	21
9	このファイルの位置づけ	21

1 対象

項目	内容
ファイル	mpc_solarcar/speed_command_bridge_node.py
ソース SHA-256	a91dadd4b230c80c827cd7002b487ced7c6d7262daed711e6ca5809536f74b67
種別	Python
区分	runtime node
役割	planner/speed_cmd と drive_mode を受け、起動直後や system state を見ながら安全な出力 topic/UDP に整形する。
起動文脈	実機へ出る直前のガード層。

2 このファイルを読む理由

planner/speed_cmd と drive_mode を受け、起動直後や system state を見ながら安全な出力 topic/UDP に整形する。

- rate limiter と command gate を持つ。
- planner の生指令をそのまま実車へは出さない。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/live_launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/solar_preflight_logic.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/signal_utils.py
- mpc_solarcar/solar_preflight_logic.py
- mpc_solarcar/telemetry_protocol.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from .telemetry_protocol import utc_iso_now

class SpeedCommandBridgeNode(Node):
    def __init__(self) -> None:
```

```

super().__init__("speed_command_bridge_node")
self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")
self.declare_parameter("udp_enabled", False)
self.declare_parameter("udp_host", "127.0.0.1")
self.declare_parameter("udp_port", 50050)
self.declare_parameter("publish_rate_hz", 5.0)
self.declare_parameter("input_timeout_sec", 3.0)
self.declare_parameter("safe_speed_kmh", 0.0)
self.declare_parameter("startup_hold_sec", 2.0)
self.declare_parameter("filter_tau_sec", 1.0)
self.declare_parameter("accel_limit_kmhps", 1.2)
self.declare_parameter("decel_limit_kmhps", 3.5)
self.declare_parameter("speed_deadband_kmh", 0.1)
self.declare_parameter("speed_quantize_step_kmh", 0.1)
self.declare_parameter("max_output_speed_kmh", 120.0)
self.declare_parameter("drive_mode_min_hold_sec", 5.0)

```

5 主要構造の読み取り

主要クラスは SpeedCommandBridgeNode。主要関数は main。ROS パラメータ宣言は 18 件。ROS I/O は publisher 2 件、subscription 4 件。

5.1 主要クラス・関数

行	種別	名前	補足
17	class	SpeedCommandBridgeNode	bases: Node
18	function	__init__	args: self
89	function	_on_speed_cmd	args: self, msg
93	function	_on_upper_speed_cmd	args: self, msg
96	function	_on_drive_mode	args: self, msg
100	function	_on_system_state	args: self, msg
104	function	_step	args: self
162	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
17	クラス SpeedCommandBridgeNode を定義する。
162	関数 main を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	from __future__ _import annotations	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。

3	<code>import json</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L153。
4	<code>import socket</code>	socket モジュールを利用するため。このファイル内での主な使用位置は L73。
5	<code>import time</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L74, L91, L98, L102, L105, L143。
7	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L163, L165, L167。
8	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L17。
10	<code>from std_msgs. msg import Float32, String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L66, L67, L68, L69, L70, L71, L89, L93, ...。
12	<code>from signal_ utils import SmoothRateLimiter</code>	signal_utils.py から SmoothRateLimiter を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/signal_utils.py。このファイル内での主な使用位置は L54。
13	<code>from solar_preflight_ logic import evaluate_ command_gate</code>	preflight 判定ロジックから evaluate_command_gate を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_preflight_logic.py。このファイル内での主な使用位置は L107。
14	<code>from telemetry_ protocol import utc_iso_now</code>	telemetry_protocol.py から utc_iso_now を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/telemetry_protocol.py。このファイル内での主な使用位置は L144。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super(). __init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self. step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscription などを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executor の実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_kmh /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.12 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L17 クラス SpeedCommandBridgeNode

- 行範囲: L17–L159
- 所属: module 直下
- 定義: SpeedCommandBridgeNode(bases=Node)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_on_speed_cmd` を定義する。
3. 関数 `_on_upper_speed_cmd` を定義する。
4. 関数 `_on_drive_mode` を定義する。
5. 関数 `_on_system_state` を定義する。
6. 関数 `_step` を定義する。

代表コード断片

```
class SpeedCommandBridgeNode(Node):
    def __init__(self) -> None:
        super().__init__("speed_command_bridge_node")
        self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
        self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")
        self.declare_parameter("udp_enabled", False)
        self.declare_parameter("udp_host", "127.0.0.1")
        self.declare_parameter("udp_port", 50050)
        self.declare_parameter("publish_rate_hz", 5.0)
```

```

self.declare_parameter("input_timeout_sec", 3.0)
self.declare_parameter("safe_speed_kmh", 0.0)
self.declare_parameter("startup_hold_sec", 2.0)
self.declare_parameter("filter_tau_sec", 1.0)
self.declare_parameter("accel_limit_kmhps", 1.2)
self.declare_parameter("decel_limit_kmhps", 3.5)
self.declare_parameter("speed_deadband_kmh", 0.1)
self.declare_parameter("speed_quantize_step_kmh", 0.1)
self.declare_parameter("max_output_speed_kmh", 120.0)
self.declare_parameter("drive_mode_min_hold_sec", 5.0)
self.declare_parameter("require_system_running", True)
self.declare_parameter("system_state_timeout_sec", 2.5)

self.output_speed_topic = str(self.get_parameter("output_speed_topic").value)
self.output_drive_mode_topic = str(self.get_parameter("output_drive_mode_topic").
value)
self.udp_enabled = bool(self.get_parameter("udp_enabled").value)
self.udp_host = str(self.get_parameter("udp_host").value)
self.udp_port = int(self.get_parameter("udp_port").value)
self.publish_rate_hz = max(1.0, float(self.get_parameter("publish_rate_hz").value))
self.input_timeout_sec = max(0.1, float(self.get_parameter("input_timeout_sec").
value))
self.safe_speed_kmh = float(self.get_parameter("safe_speed_kmh").value)
self.startup_hold_sec = max(0.0, float(self.get_parameter("startup_hold_sec").value)
)
self.drive_mode_min_hold_sec = max(0.0, float(self.get_parameter("
drive_mode_min_hold_sec").value))
self.require_system_running = bool(self.get_parameter("require_system_running").
value)
self.system_state_timeout_sec = max(
    0.1, float(self.get_parameter("system_state_timeout_sec").value)
...

```

7.2 L18 関数 SpeedCommandBridgeNode.__init__

- 行範囲: L18–L87
- 所属: SpeedCommandBridgeNode
- 定義: `__init__(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: SmoothRateLimiter, `__init__`, `bool`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `get_logger`, `get_parameter`, `info`, `int`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.on_drive_mode`, `self.on_speed_cmd`, `self.on_system_state`, `self.on_upper_speed_cmd`, `self.step`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.get_logger`, `self.get_parameter`, `self.limiter`, `self.output_drive_mode_topic`, `self.output_speed_topic`, `self.publish_rate_hz`, `self.safe_speed_kmh`, `self.start_time`, `self.udp_enabled`
- 更新する主なインスタンス属性: `self.applied_drive_mode`, `self.drive_mode_min_hold_sec`, `self.input_timeout_sec`, `self.last_drive_mode`, `self.last_drive_mode_change`, `self.last_gate_reason`, `self.last_mode_input_time`, `self.last_speed_cmd`, `self.last_speed_input_time`, `self.last_system_state_time`, `self.last_udp_error_log_time`, `self.last_upper_speed_cmd`, `self.limiter`, `self.mode_pub`, `self.output_drive_mode_topic`, `self.output_speed_topic`, `self.publish_rate_hz`, `self.require_system_running`, `self.safe_speed_kmh`, `self.sock`, `self.speed_pub`, `self.start_time`, `self.startup_hold_sec`, `self.system_state`

- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。
13. `self.declare_parameter(...)` を実行する。
14. `self.declare_parameter(...)` を実行する。
15. `self.declare_parameter(...)` を実行する。
16. `self.declare_parameter(...)` を実行する。
17. `self.declare_parameter(...)` を実行する。
18. `self.declare_parameter(...)` を実行する。
19. `self.declare_parameter(...)` を実行する。
20. `self.output_speed_topic` に `str(self.get_parameter('output_speed_topic').value)` の結果を代入する。
21. `self.output_drive_mode_topic` に `str(self.get_parameter('output_drive_mode_topic').value)` の結果を代入する。
22. `self.udp_enabled` に `bool(self.get_parameter('udp_enabled').value)` の結果を代入する。
23. `self.udp_host` に `str(self.get_parameter('udp_host').value)` の結果を代入する。
24. `self.udp_port` に `int(self.get_parameter('udp_port').value)` の結果を代入する。
25. `self.publish_rate_hz` に `max(1.0, float(self.get_parameter('publish_rate_hz').value))` の結果を代入する。
26. `self.input_timeout_sec` に `max(0.1, float(self.get_parameter('input_timeout_sec').value))` の結果を代入する。

27. `self.safe_speed_kmh` に `float(self.get_parameter('safe_speed_kmh').value)` の結果を代入する。
28. `self.startup_hold_sec` に `max(0.0, float(self.get_parameter('startup_hold_sec').value))` の結果を代入する。
29. `self.drive_mode_min_hold_sec` に `max(0.0, float(self.get_parameter('drive_mode_min_hold_sec').value))` の結果を代入する。
30. `self.require_system_running` に `bool(self.get_parameter('require_system_running').value)` の結果を代入する。
31. `self.system_state_timeout_sec` に `max(0.1, float(self.get_parameter('system_state_timeout_sec').value))` の結果を代入する。
32. `self.limiter` に `SmoothRateLimiter(min_value=0.0, max_value=float(self.get_parameter('max_output_speed_kmh').value), tau_sec=float(self.get_parameter('filter_tau_sec').value), rise_rate=float(self.get_parameter('accel_limit_kmhps').value), fall_rate=float(self.get_parameter('decel_limit_kmhps').value), deadband=float(self.get_parameter('speed_deadband_kmh').value), quantize_step=float(self.get_parameter('speed_quantize_step_kmh').value), initial_value=self.safe_speed_kmh)` の結果を代入する。
33. `self.limiter.reset(...)` を実行する。
34. `self.speed_pub` に `self.create_publisher(Float32, self.output_speed_topic, 10)` の結果を代入する。
35. `self.mode_pub` に `self.create_publisher(String, self.output_drive_mode_topic, 10)` の結果を代入する。
36. `self.create_subscription(...)` を実行する。
37. `self.create_subscription(...)` を実行する。
38. `self.create_subscription(...)` を実行する。
39. `self.create_subscription(...)` を実行する。
40. `self.sock` に `socket.socket(socket.AF_INET, socket.SOCK_DGRAM) if self.udp_enabled else None` の結果を代入する。
41. `self.start_time` に `time.monotonic()` の結果を代入する。
42. `self.last_speed_input_time` に `None` の結果を代入する。
43. `self.last_mode_input_time` に `None` の結果を代入する。
44. `self.last_system_state_time` に `None` の結果を代入する。
45. `self.system_state` に `'STARTING'` の結果を代入する。
46. `self.last_speed_cmd` に `self.safe_speed_kmh` の結果を代入する。
47. `self.last_upper_speed_cmd` に `self.safe_speed_kmh` の結果を代入する。
48. `self.last_drive_mode` に `'stop'` の結果を代入する。
49. `self.applied_drive_mode` に `'stop'` の結果を代入する。
50. `self.last_drive_mode_change` に `self.start_time` の結果を代入する。
51. `self.last_udp_error_log_time` に `float('-inf')` の結果を代入する。
52. `self.last_gate_reason` に `'startup_hold'` の結果を代入する。
53. `self.timer` に `self.create_timer(1.0 / self.publish_rate_hz, self._step)` の結果を代入する。
54. `self.get_logger().info(...)` を実行する。

代表コード断片

```
def __init__(self) -> None:
    super().__init__("speed_command_bridge_node")
    self.declare_parameter("output_speed_topic", "/vehicle/speed_cmd_kmh")
    self.declare_parameter("output_drive_mode_topic", "/vehicle/drive_mode_cmd")
    self.declare_parameter("udp_enabled", False)
    self.declare_parameter("udp_host", "127.0.0.1")
    self.declare_parameter("udp_port", 50050)
    self.declare_parameter("publish_rate_hz", 5.0)
    self.declare_parameter("input_timeout_sec", 3.0)
    self.declare_parameter("safe_speed_kmh", 0.0)
    self.declare_parameter("startup_hold_sec", 2.0)
    self.declare_parameter("filter_tau_sec", 1.0)
    self.declare_parameter("accel_limit_kmhps", 1.2)
    self.declare_parameter("decel_limit_kmhps", 3.5)
    self.declare_parameter("speed_deadband_kmh", 0.1)
    self.declare_parameter("speed_quantize_step_kmh", 0.1)
    self.declare_parameter("max_output_speed_kmh", 120.0)
    self.declare_parameter("drive_mode_min_hold_sec", 5.0)
    self.declare_parameter("require_system_running", True)
    self.declare_parameter("system_state_timeout_sec", 2.5)

    self.output_speed_topic = str(self.get_parameter("output_speed_topic").value)
    self.output_drive_mode_topic = str(self.get_parameter("output_drive_mode_topic").value)
    self.udp_enabled = bool(self.get_parameter("udp_enabled").value)
    self.udp_host = str(self.get_parameter("udp_host").value)
    self.udp_port = int(self.get_parameter("udp_port").value)
    self.publish_rate_hz = max(1.0, float(self.get_parameter("publish_rate_hz").value))
    self.input_timeout_sec = max(0.1, float(self.get_parameter("input_timeout_sec").value))
    self.safe_speed_kmh = float(self.get_parameter("safe_speed_kmh").value)
    self.startup_hold_sec = max(0.0, float(self.get_parameter("startup_hold_sec").value))
    self.drive_mode_min_hold_sec = max(0.0, float(self.get_parameter("drive_mode_min_hold_sec").value))
    self.require_system_running = bool(self.get_parameter("require_system_running").value)
    self.system_state_timeout_sec = max(0.1, float(self.get_parameter("system_state_timeout_sec").value))
    ...
```

7.3 L89 関数 SpeedCommandBridgeNode._on_speed_cmd

- 行範囲: L89–L91
- 所属: SpeedCommandBridgeNode
- 定義: `_on_speed_cmd(self, msg: Float32) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `monotonic`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.last_speed_cmd`, `self.last_speed_input_time`

- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.last_speed_cmd` に `float(msg.data)` の結果を代入する。
2. `self.last_speed_input_time` に `time.monotonic()` の結果を代入する。

代表コード断片

```
def _on_speed_cmd(self, msg: Float32) -> None:
    self.last_speed_cmd = float(msg.data)
    self.last_speed_input_time = time.monotonic()
```

7.4 L93 関数 `SpeedCommandBridgeNode._on_upper_speed_cmd`

- 行範囲: L93–L94
- 所属: `SpeedCommandBridgeNode`
- 定義: `_on_upper_speed_cmd(self, msg: Float32) -> None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.last_upper_speed_cmd`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.last_upper_speed_cmd` に `float(msg.data)` の結果を代入する。

代表コード断片

```
def _on_upper_speed_cmd(self, msg: Float32) -> None:
    self.last_upper_speed_cmd = float(msg.data)
```

7.5 L96 関数 SpeedCommandBridgeNode._on_drive_mode

- 行範囲: L96–L98
- 所属: SpeedCommandBridgeNode
- 定義: `_on_drive_mode(self, msg:String) ->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `monotonic`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.last_drive_mode`, `self.last_mode_input_time`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.last_drive_mode` に `str(msg.data or 'stop')` の結果を代入する。
2. `self.last_mode_input_time` に `time.monotonic()` の結果を代入する。

代表コード断片

```
def _on_drive_mode(self, msg: String) -> None:
    self.last_drive_mode = str(msg.data or "stop")
    self.last_mode_input_time = time.monotonic()
```

7.6 L100 関数 SpeedCommandBridgeNode._on_system_state

- 行範囲: L100–L102
- 所属: SpeedCommandBridgeNode
- 定義: `_on_system_state(self, msg:String) ->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `monotonic`, `str`, `strip`, `upper`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.last_system_state_time`, `self.system_state`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.system_state` に `str(msg.data or "").strip().upper()` の結果を代入する。
2. `self.last_system_state_time` に `time.monotonic()` の結果を代入する。

代表コード断片

```
def _on_system_state(self, msg: String) -> None:
    self.system_state = str(msg.data or "").strip().upper()
    self.last_system_state_time = time.monotonic()
```

7.7 L104 関数 `SpeedCommandBridgeNode._step`

- 行範囲: L104–L159
- 所属: `SpeedCommandBridgeNode`
- 定義: `_step(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `String`, `dumps`, `encode`, `error`, `evaluate_command_gate`, `float`, `get_logger`, `monotonic`, `publish`, `sendto`, `time`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `gate`, `mode_is_fresh`, `now`, `payload`, `requested_mode`, `speed_out`, `speed_target`
- 読み取る主なインスタンス属性: `self.applied_drive_mode`, `self.drive_mode_min_hold_sec`, `self.get_logger`, `self.input_timeout_sec`, `self.last_drive_mode`, `self.last_drive_mode_change`, `self.last_gate_reason`, `self.last_mode_input_time`, `self.last_speed_cmd`, `self.last_speed_input_time`, `self.last_system_state_time`, `self.last_udp_error_log_time`, `self.last_upper_speed_cmd`, `self.limiter`, `self.mode_pub`, `self.require_system_running`, `self.safe_speed_kmh`, `self.sock`, `self.speed_pub`, `self.start_time`, `self.startup_hold_sec`, `self.system_state`, `self.system_state_timeout_sec`, `self.udp_host`
- 更新する主なインスタンス属性: `self.applied_drive_mode`, `self.last_drive_mode_change`, `self.last_gate_reason`, `self.last_udp_error_log_time`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. now に `time.monotonic()` の結果を代入する。
2. `speed_target` に `self.safe_speed_kmh` の結果を代入する。
3. `gate` に `evaluate_command_gate(elapsed_sec=now - self.start_time, speed_input_age_sec=None if self.last_speed_input_time is None else now - self.last_speed_input_time, system_state=self.system_state, system_state_age_sec=None if self.last_system_state_time is None else now - self.last_system_state_time, startup_hold_sec=self.startup_hold_sec, input_timeout_sec=self.input_timeout_sec, system_state_timeout_sec=self.system_state_timeout_sec, require_system_running=self.require_system_running)` の結果を代入する。
4. `self.last_gate_reason` に `gate.reason` の結果を代入する。
5. 条件 `gate.allowed` を判定し、真なら内部処理を行う。
6. `speed_target` に `self.last_speed_cmd` の結果を代入する。
7. `speed_out` に `float(self.limiter.update(speed_target, now=now))` の結果を代入する。
8. `mode_is_fresh` に `self.last_mode_input_time is not None and now - self.last_mode_input_time <= self.input_timeout_sec` の結果を代入する。
9. `requested_mode` に `self.last_drive_mode if gate.allowed and mode_is_fresh else 'stop'` の結果を代入する。
10. 条件 `requested_mode == 'stop'` を判定し、真なら内部処理を行う。
11. `self.applied_drive_mode` に `'stop'` の結果を代入する。
12. `self.last_drive_mode_change` に `now` の結果を代入する。
13. 上の条件が偽の場合:
14. 条件 `requested_mode != self.applied_drive_mode and now - self.last_drive_mode_change >= self.drive_mode_min_hold_time` を判定し、真なら内部処理を行う。
15. `self.applied_drive_mode` に `requested_mode` の結果を代入する。
16. `self.last_drive_mode_change` に `now` の結果を代入する。
17. `self.speed_pub.publish(...)` を実行する。
18. `self.mode_pub.publish(...)` を実行する。
19. 条件 `self.sock is not None` を判定し、真なら内部処理を行う。
20. `payload` に `{'type': 'planner_command', 'ts_unix': time.time(), 'timestamp_utc': utc_iso_now(), 'planner': {'speed_cmd_kmh': speed_out, 'upper_speed_cmd_kmh': self.last_upper_speed_cmd, 'drive_mode': self.applied_drive_mode, 'command_gate': self.last_gate_reason}}` の結果を代入する。
21. 例外処理を伴う `try` ブロックを実行する。
22. `self.sock.sendto(...)` を実行する。
23. `OSError` を捕捉した場合:
24. 条件 `now - self.last_udp_error_log_time >= 5.0` を判定し、真なら内部処理を行う。
25. `self.get_logger().error(...)` を実行する。
26. `self.last_udp_error_log_time` に `now` の結果を代入する。

代表コード断片

```
def _step(self) -> None:
    now = time.monotonic()
    speed_target = self.safe_speed_kmh
    gate = evaluate_command_gate(
        elapsed_sec=now - self.start_time,
        speed_input_age_sec=(
            None if self.last_speed_input_time is None else now - self.
last_speed_input_time
        ),
        system_state=self.system_state,
        system_state_age_sec=(
            None if self.last_system_state_time is None else now - self.
last_system_state_time
        ),
        startup_hold_sec=self.startup_hold_sec,
        input_timeout_sec=self.input_timeout_sec,
        system_state_timeout_sec=self.system_state_timeout_sec,
        require_system_running=self.require_system_running,
    )
    self.last_gate_reason = gate.reason
    if gate.allowed:
        speed_target = self.last_speed_cmd
        speed_out = float(self.limiter.update(speed_target, now=now))

        mode_is_fresh = (
            self.last_mode_input_time is not None
            and (now - self.last_mode_input_time) <= self.input_timeout_sec
        )
        requested_mode = self.last_drive_mode if gate.allowed and mode_is_fresh else "stop"
        if requested_mode == "stop":
            self.applied_drive_mode = "stop"
            self.last_drive_mode_change = now
        elif requested_mode != self.applied_drive_mode and (now - self.
last_drive_mode_change) >= self.drive_mode_min_hold_sec:
            self.applied_drive_mode = requested_mode
            self.last_drive_mode_change = now

        self.speed_pub.publish(Float32(data=speed_out))
...
```

7.8 L162 関数 main

- 行範囲: L162–L167
- 所属: module 直下
- 定義: `main()->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `SpeedCommandBridgeNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. rclpy.init(...) を実行する。
2. node に SpeedCommandBridgeNode() の結果を代入する。
3. rclpy.spin(...) を実行する。
4. node.destroy_node(...) を実行する。
5. rclpy.shutdown(...) を実行する。

代表コード断片

```
def main() -> None:
    rclpy.init()
    node = SpeedCommandBridgeNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

7.9 設定値と入力パラメータ

行	名前	既定値・補足
20	output_speed_topic	/vehicle/speed_cmd_kmh
21	output_drive_mode_topic	/vehicle/drive_mode_cmd
22	udp_enabled	False
23	udp_host	127.0.0.1
24	udp_port	50050
25	publish_rate_hz	5.0
26	input_timeout_sec	3.0
27	safe_speed_kmh	0.0
28	startup_hold_sec	2.0
29	filter_tau_sec	1.0
30	accel_limit_kmhps	1.2
31	decel_limit_kmhps	3.5
32	speed_deadband_kmh	0.1
33	speed_quantize_step_kmh	0.1
34	max_output_speed_kmh	120.0
35	drive_mode_min_hold_sec	5.0
36	require_system_running	True
37	system_state_timeout_sec	2.5

7.10 ROS topic I/O

行	種別	topic	callback / 補足
66	publisher	self.output_speed_topic	publisher
67	publisher	self.output_drive_mode_topic	publisher
68	subscription	/planner/speed_cmd	self._on_speed_cmd
69	subscription	/planner/upper_speed_cmd	self._on_upper_speed_cmd
70	subscription	/planner/drive_mode	self._on_drive_mode
71	subscription	/system/state	self._on_system_state

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/speed_command_bridge_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。